CS 3563: Assignment 2 - Query Optimization Analysis

Asit Patel (CS24BTECH11046)
Shiva Chethan (CS24BTECH11057)
Indian Institute of Technology Hyderabad

March 16, 2026

1 Introduction and Methodology

The objective of this task was to analyze 10 specific plan switches across three queries to determine if the PostgreSQL optimizer made the switch at the optimal time. By using the `pg_hint_plan` extension, I forced the optimizer to run the original plan (P_i) on the new date (q_j), and the new plan (P_j) on the old date (q_i). By comparing their actual execution times, I classified each switch as CORRECT, EARLY, or DELAYED.

1.1 The Cache Warming Problem

Before analyzing the data, I encountered a significant testing hurdle: disk caching. The first time I ran a query, it took significantly longer because PostgreSQL had to fetch data blocks directly from the disk. Running the competing plan immediately after resulted in artificially fast times because the data was already cached in RAM.

To ensure my execution time comparisons were fair and consistent, I modified my Python testing script to run `EXPLAIN (ANALYZE, FORMAT JSON)` twice for every test. The first run acted as a "cache warmer," and I only recorded the execution time from the second run. This made the runtime margins mathematically accurate and removed hardware bias.

2 Experimental results, Observations and Explanations

Writing the `pg_hint_plan` strings started off relatively straightforward for Queries 1 and 2, but became incredibly complex for Query 3. To verify my hints were working, I wrote a Python

function to extract the physical operator tree from the JSON output and compare it to the target plan. During this verification process, I observed several anomalies in how the PostgreSQL optimizer behaves when forced into specific paths.

2.1 Query 1: Basic Scans and Parallelism

Switch ($S_{i,j}$)	$RT(P_i, q_i)$	$RT(P_j, q_j)$	$RT(P_i, q_j)$	$RT(P_j, q_i)$	**Classification**
1992-01-10 to 1992-01-11	14.487	25.354	16.660	15.555	**EARLY**
1992-08-21 to 1992-08-22	18867.697	10829.757	18860.678	11227.871	**DELAYED**

For Query 1, the structural changes were simple.

- **Switch 1:** The optimizer added a `Gather` node to enable parallel execution. I forced this using `/*+ Parallel(lineitem 0) */` to simulate the serial plan, and `/*+ Parallel(lineitem 4) */` to force the parallel working.
- **Switch 2:** The scan method changed from a `Bitmap Heap Scan` to a full `Seq Scan`, which I controlled using `/*+ BitmapScan(lineitem) */` and `/*+ SeqScan(lineitem) */`.

2.2 Query 2: The Global Flag Interference Anomaly

Switch ($S_{i,j}$)	$RT(P_i, q_i)$	$RT(P_j, q_j)$	$RT(P_i, q_j)$	$RT(P_j, q_i)$	**Classification**
1993-03-11 to 1993-03-12	18026.121	15513.506	18581.758	15720.659	**DELAYED**
1993-04-28 to 1993-04-29	15599.738	15611.783	15542.473	15586.109	**DELAYED**
1996-11-22 to 1996-11-23	16097.162	11243.989	15520.368	14478.352	**DELAYED**

```
P_i: /*+ Set(enable_hashagg off) */
P_j: /*+ Set(enable_sort off) */
```

The Anomaly:

When I used `Set(enable_sort off)` to force the `HashAgg` on the old date, my tree-comparison script reported a mismatch. Upon manual inspection of the JSON, I found that the native `HashAgg` plan still used a tiny `Sort` node at the very top of the tree just to order the final 5 output rows. Because I had globally banned sorting, PostgreSQL dropped that final `Sort` node entirely. The core `HashAgg` algorithm was successfully forced, making the execution times perfectly valid, but it demonstrated that using `Set()` flags acts as a blunt instrument rather than a precise control.

Switch ($S_{i,j}$)	$RT(P_i, q_i)$	$RT(P_j, q_j)$	$RT(P_i, q_j)$	$RT(P_j, q_i)$	**Classification**
1992-02-28 to 1992-02-29	1149.913	412.027	617.025	483.896	**DELAYED**
1993-04-06 to 1993-04-07	16956.405	14248.311	17020.069	14303.927	**DELAYED**
1998-08-28 to 1998-08-29	15912.786	12480.909	15887.603	12526.224	**DELAYED**
1998-09-19 to 1998-09-20	862.008	897.586	815.871	886.264	**EARLY**
1998-11-30 to 1998-12-01	5.728	0.011	0.011	1.667	**DELAYED**

2.3 Query 3: Optimizer Drift and Nuclear Hints

Query 3 joins three tables (`lineitem`, `orders`, `customer`) and caused the most significant challenges. For the later switches in 1998, simply telling PostgreSQL what join order to use was not enough.

Initially, I tried hints like:

```
/*+ Leading(o c l) NestLoop(o c l) */
```

However, the database consistently rejected the structure.

The Optimizer Drift Anomaly:

When you force PostgreSQL into a join order it mathematically estimates to be "expensive," its internal cost calculations compensate by silently swapping out the leaf-level scan types. For example, it would arbitrarily drop an `Index Scan` and use a `Seq Scan` on the `orders` table to try and save time. Because the scan type changed, the resulting plan tree did not match my target plan.

The Solution (Fully Explicit Hints):

To stop the optimizer from drifting, I realized I had to write extremely long, highly specific hints that locked down every single node in the tree. I had to flatten the `Leading` syntax to avoid parsing bugs, define the join methods for every step, and explicitly hardcode the scan methods for all three tables.

```
/*+ Leading(l o c) BitmapScan(l) IndexScan(o) SeqScan(c) NestLoop(l o) HashJoin(l o c) */
```

By completely removing all freedom from the optimizer, I was finally able to achieve 100% structural matches between my forced plans and the native target plans.

2.4 The Dropped Gather Node Anomaly

For the final switch of Query 3 (1998-11-30), I hit one last anomaly. Because the date was at the absolute end of the dataset, there was practically zero data passing the `WHERE` filters.

When I forced the plans here, PostgreSQL silently removed the parallel `Gather` nodes from the execution tree. Even though my hints perfectly forced the core `Nested Loops` and `Index Scans`,

the database engine dynamically decided that spinning up parallel worker threads to process a single row of data was a massive waste of resources. This was a smart override by the execution engine, and since the algorithmic access paths matched exactly, the timing data was valid for final classification.

3 Conclusion

The experimental data clearly highlights a systemic behavior in the PostgreSQL cost-based optimizer: inertia.

The vast majority of the plan switches across all 3 queries were classified as **DELAYED**.

The optimizer is highly conservative. It systematically underestimates the real-world efficiency of Sequential Scans and Hash Joins on growing datasets, preferring to maintain Nested Loops and Index Scans for months (or years) longer than it mathematically should.